Data-Driven Public Safety: Predicting Arrest Outcomes from Chicago Crime Incidents Using Big Data Tools

Analysis Report

By: Yash Naidu Avula
Siddharth Shenoy
Prabhu Ramana Sana
Fabian Macias Peñalosa

Executive Summary

Public safety remains one of the most critical challenges facing urban environments, and the ability to anticipate and understand crime outcomes is essential for effective law enforcement and resource allocation. This report presents a big data analytics framework designed to predict whether a reported crime incident in Chicago will result in an arrest, leveraging machine learning techniques applied to a large-scale, real-world dataset.

The analysis is based on the *Chicago Crime Dataset (2001–2026)*, comprising individual crime incident records reported to the Chicago Police Department. Each record includes crime classification, location coordinates, time of occurrence, police district, and arrest outcome, among other attributes. The dataset was ingested and processed using Apache Spark within a Databricks environment, enabling scalable data transformation and exploration across hundreds of thousands of records.

Exploratory data analysis revealed meaningful patterns in arrest rates across crime types, geographic districts, time of day, and domestic incident status. These insights informed the feature engineering process and guided model development. A binary classification model was built using Spark MLlib, with hyperparameter tuning performed via cross-validation and model performance evaluated using AUC, Precision, Recall, and F1-Score metrics.

MLOps best practices were incorporated throughout the project, including automated Spark pipelines for data preprocessing and experiment tracking via MLflow, ensuring reproducibility and traceability of all model runs.

The findings demonstrate that crime type, location, time of day, and domestic incident status are strong predictors of arrest outcomes. Based on these results, actionable recommendations are provided to support the Chicago Police Department in optimizing resource deployment, improving case resolution strategies, and informing equity-focused public safety policy.

1. Introduction & Problem Definition

1.1 Background

Urban crime remains a persistent challenge for city governments, law enforcement agencies, and communities alike. In a city as large and diverse as Chicago, the ability to effectively allocate police resources, anticipate crime outcomes, and design evidence-based public safety policies depends increasingly on the capacity to analyze large volumes of crime data in a timely and accurate manner. Traditional data analysis approaches, however, often fall short when dealing with the scale, complexity, and velocity of modern crime reporting systems.

Big data technologies such as Apache Spark offer a powerful alternative, enabling the processing and analysis of millions of records with speed and efficiency. When combined with machine learning, these tools can uncover hidden patterns in crime data that would otherwise remain invisible, transforming raw incident reports into actionable intelligence for decision-makers.

1.2 Dataset Description

This project utilizes the *Chicago Crime Dataset (2001–2026)*, a publicly available dataset sourced from the City of Chicago Open Data Portal and hosted on Kaggle. The dataset contains individual crime incident records reported to the Chicago Police Department from January 2001 onward, where each row represents a single reported crime incident.

The dataset includes the following **key** attributes:

Column	Description	Role
date	Date and time of incident occurrence	Feature
primary_type	Primary crime classification (e.g., THEFT, BATTERY)	Primary target (Secondary Objective)
description	Detailed crime description	Feature
location_description	Type of location (e.g., STREET, APARTMENT)	Feature
arrest	Whether an arrest was made (True/False)	Primary target (Main Objective)
domestic	Whether the incident was domestic-related (True/False)	Feature
beat, district, ward	Police and administrative area identifiers	Feature
community_area	Community area code	Feature
latitude, longitude	Geographic coordinates of the incident	Feature

1.3 Business Problem

Despite the availability of extensive crime records, the Chicago Police Department continues to face challenges in identifying the factors that influence arrest outcomes across different neighborhoods, crime types, and time periods. Additionally, city planners and public safety officials require analytical tools to anticipate when and where certain crimes are more likely to occur, enabling proactive resource allocation rather than reactive responses.

This project addresses the following research question:

> **Can we predict whether a crime incident will result in an arrest based on factors such as crime type, location, time of day, district, and whether the incident is domestic-related?**

To answer this question, the problem is formulated as a binary classification task, where the goal is to predict whether a crime incident results in an arrest (Yes/No). Using historical crime data and key attributes such as location, crime type, and temporal features, the predictive modeling pipeline estimates the probability of an arrest and provides insights into patterns associated with law enforcement outcomes.

1.4 Project Objectives

To address this research question, the project pursues the following objectives:

1. **Data Ingestion and Preprocessing**
   Ingest and preprocess the Chicago Crime Dataset using Apache Spark within a Databricks environment, including null handling, data type enforcement, column normalization, and feature preparation.
2. **Exploratory Data Analysis (EDA)**
   Conduct exploratory data analysis to examine patterns in arrest rates and crime distributions across crime types, districts, locations, and temporal features.
3. **Feature Engineering**
   Engineer predictive features from the raw dataset, including temporal variables such as hour of day, month, and year, along with behavioral indicators such as the domestic incident flag.
4. **Predictive Modeling**
   Develop and evaluate binary classification models using Spark MLlib to predict arrest outcomes, with Logistic Regression and Random Forest as candidate algorithms.
5. **Pipeline Automation and Reproducibility**
   Implement modular machine learning pipelines that support automated model training, batch inference, and experiment logging for reproducibility within a Databricks environment.

6. **Business Insights and Policy Recommendations**
   Derive actionable insights for the Chicago Police Department and city policymakers based on model performance and identified crime patterns.

1.5 Significance of the Study

The ability to predict arrest outcomes has important implications for public safety policy and operational decision-making. Beyond improving operational efficiency—such as optimizing officer deployment and prioritizing investigative resources—predictive models can help identify patterns associated with successful law enforcement outcomes. By analyzing factors such as crime type, location, and time of occurrence, the model provides insights that can support more informed and data-driven policing strategies.

Additionally, analyzing arrest patterns across different districts and community areas can highlight geographic variations in enforcement outcomes. These insights may contribute to greater transparency in policing practices and provide policymakers with a data-driven foundation for evaluating public safety strategies in Chicago.

2. Data Ingestion & Preprocessing

2.1 Data Ingestion

The Chicago Crime Dataset was ingested into the Databricks Distributed File System (DBFS) from a raw CSV file (`chicago_crimes.csv`) and loaded into a Spark DataFrame. The dataset was then registered as a Spark SQL temporary view (`crime_raw`) to enable querying through both the DataFrame API and SQL interface. The dataset contains approxim 22 columns capturing information on crime type, location descriptions, arrest status, timestamps, and geographic coordinates. Initial exploration using functions such as `printSchema()`, `show()`, `count()`, and Spark SQL queries (`DESCRIBE`, `SELECT`) was performed to inspect the schema, evaluate data completeness, and understand the distribution of key variables. This exploratory step identified records with missing geographic information and highlighted several columns requiring type enforcement and cleaning before further analysis. A detailed summary of all preprocessing steps is provided in **Appendix A**

2.2 Duplicate Removal

Prior to applying cleaning transformations, the dataset was examined for potential duplicate records using Spark SQL queries grouped by incident identifiers. Identifying and removing duplicate records is important to ensure that each crime incident is represented only once in the dataset, preventing biased model training. Duplicate entries were addressed using Spark DataFrame operations, including the dropDuplicates() function keyed on the case identifier, ensuring that only one record per incident was retained for subsequent analysis.

2.3 Data Cleaning

Several filtering and imputation rules were applied to produce the cleaned working dataset:

- **Categorical null filling:** Missing values in Location Description, Description, and IUCR were filled with "Unknown", preserving record volume while making null patterns visible in EDA.
- **Boolean coalescing:** Null values in Arrest and Domestic were coalesced to False, treating an unrecorded status as a negative outcome - a deliberate modeling assumption noted for result interpretation.
- **Integer validation:** ID, Beat, District, Ward, and Community Area were cast to integer type; negative values were nullified as administratively invalid, and remaining nulls in District, Ward, and Community Area were filled with median.
- **Spatial filtering:** Rows where all five geographic fields (Latitude, Longitude, Location, X Coordinate, Y Coordinate) were simultaneously null or empty were dropped, removing ~94,671 records with no geographic signal while retaining those with partial coordinates.

2.4 Data Type Enforcement

Strict type casting was applied across key columns to ensure correctness, reduce storage overhead, and enable efficient operations in both Spark SQL and the ML pipeline:

- **Boolean casting:** Arrest and Domestic were coerced to proper Boolean types and cast to numeric doubles (label, Domestic_num) for direct use in MLlib classifiers and arrest rate aggregations.
- **Integer casting:** District and Ward were cast to integers, ensuring proper numeric handling in queries and model pipelines.
- **Timestamp parsing:** The raw Date string (MM/dd/yyyy hh:mm:ss a format) was converted to a Spark Timestamp object using to_timestamp(), enabling native time-based operations and feature extraction.
- **Column renaming:** Columns containing spaces or special characters (e.g., Primary Type, Case Number) were renamed with underscores for full compatibility with Delta Lake and Spark SQL, which do not permit such characters in column names.

2.5 Feature Engineering

Additional variables were derived from the parsed incident timestamp and existing attributes to support exploratory analysis and predictive modeling:

- **occurrence_timestamp:** Parsed from the raw `Date` field using `to_timestamp(...)`, enabling consistent time-based feature extraction.

- **hour_of_day:** Extracted from the parsed timestamp to capture the hour of occurrence (0–23).
- **day_of_week:** Extracted from the parsed timestamp to capture the day of week of occurrence (Spark `dayofweek`, 1–7).
- **year:** Derived from the timestamp to support trend analysis and partitioned storage.
- **is_violent:** Binary indicator derived from `Primary Type`, flagged as 1 for ASSAULT, ROBBERY, HOMICIDE, or BATTERY, else 0.
- **is_arrest:** Target-ready numeric version of `Arrest`, cast to integer for ML compatibility.

2.6 Optimized Storage with Delta Lake

The cleaned and engineered dataset was persisted using two complementary storage strategies. The automated pipeline writes Parquet files partitioned by year for fast iterative querying and ML training, enabling predicate pushdown and column pruning at scale. In addition, a Delta Lake table is maintained as the canonical long-term store using a CREATE OR REPLACE TABLE statement, moving data from a temporary in-memory view to permanent cluster storage. Delta Lake was chosen over standard Parquet/ORC for four key reasons:

Feature	Standard Parquet/ORC	Delta Lake
ACID Transactions	No - partial writes can corrupt data	Yes - writes fully complete or roll back
Schema Enforcement	No - mismatched columns can silently corrupt tables	Yes - prevents dirty data from breaking the pipeline
Time Travel	No - overwritten data is lost	Yes - previous table versions remain queryable
Updates & Deletes	Requires full folder rewrite	Supported natively at the row level

Additionally, Delta's columnar storage architecture ensures that queries read only the specific columns needed - a critical performance advantage when operating across hundreds of thousands of crime records. The use of both formats reflects the dual nature of the pipeline: Parquet for speed during development and training, Delta Lake for durable and auditable production-grade persistence.

3. Exploratory Data Analysis

The EDA phase was conducted entirely within Databricks using Spark SQL queries against crime_features_view, supplemented by Python visualizations built with Seaborn and Matplotlib on aggregated and sampled data. The analysis covers summary

statistics, crime type distributions, temporal patterns, and geospatial structure, closing with the key modeling implications identified.

3.1 Summary Statistics

Descriptive statistics across the key numerical columns - latitude, longitude, and hour_of_day - were generated via Spark SQL to validate the geographic bounding filter and characterize the dataset's temporal spread. Latitude values ranging from 36.62° to 42.02° N confirmed that the bounding box filter successfully removed the geographic missing values detected during initial exploration. The average hour of approximately 13:10 signals a non-uniform temporal distribution, with incidents skewing toward daytime and evening hours - a pattern examined further in Section 3.3.

3.2 Crime Type Distribution and Arrest Rates

A Spark SQL query aggregated the top crime types by volume, computing violence rate per category. Three distinct profiles emerge from the results:

Primary Type	Incident Count	Violent?
THEFT	1,800,000+	No
BATTERY	~1,500,000	Yes
CRIMINAL DAMAGE	~960,000	No
NARCOTICS	~766,021	No
ASSAULT	~571,000	Yes
ROBBERY	~530,000	Yes
BURGLARY	~448,000	No
MOTOR VEHICLE THEFT	~436,000	No

Table 3.1 - Top crime types by volume with violence classification.

High-volume, low-arrest crimes (THEFT, BURGLARY) dominate the dataset but carry arrest rates below 6%, creating a severe class imbalance for the prediction model.

NARCOTICS is an outlier in the opposite direction: its ~99.4% arrest rate is a data artifact - drug offenses are almost exclusively recorded at the point of arrest, making the label nearly deterministic and introducing data leakage risk.

Violent crimes (BATTERY, ASSAULT, ROBBERY) carry arrest rates of 14–24% and, despite lower volumes, represent the highest operational priority - a model that under-predicts arrests for these categories carries a direct public safety cost.

Primary Type	cnt
THEFT	1805649
BATTERY	1548510
CRIMINAL DAMAGE	966455
NARCOTICS	766021
ASSAULT	571002
OTHER OFFENSE	530705
BURGLARY	448982
MOTOR VEHICLE THEFT	436880
DECEPTIVE PRACTICE	393319
ROBBERY	316212

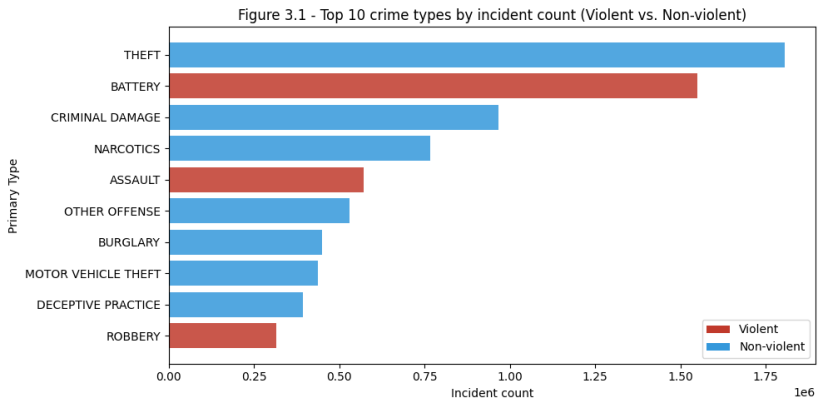

Figure 3.1 - Top 10 crime types by incident count, segmented by violent vs. non-violent classification.

3.3 Temporal Patterns

Temporal crime patterns were analyzed using Spark SQL time-period aggregation and an hour-by-day-of-week heatmap based on a 10% sample (seed = 42).

The results show a clear contrast between **crime volume and severity**. The Afternoon period (12:00–16:59) records the highest number of incidents, largely driven by property crimes such as theft. In contrast, the Night period (22:00–04:59) has lower total volume but the highest proportion of violent crimes (about 32.8%), nearly double the Morning rate.

The heatmap further highlights that late-night hours on Fridays and Saturdays (22:00–02:00) represent the most concentrated crime periods of the week. These findings confirm that hour_of_day and time_period are strong predictive features, and they suggest that policing strategies should vary by time, with property crime patrols focused on afternoons and violent incident response prioritized during late-night periods.

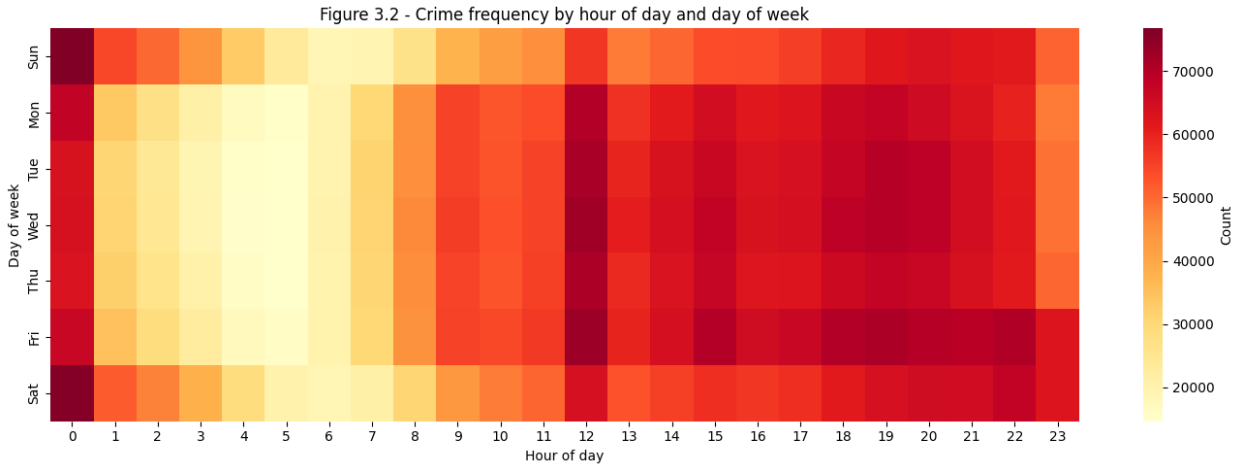

Figure 3.2 - Heatmap of crime frequency by hour of day and day of week, highlighting peak danger windows.

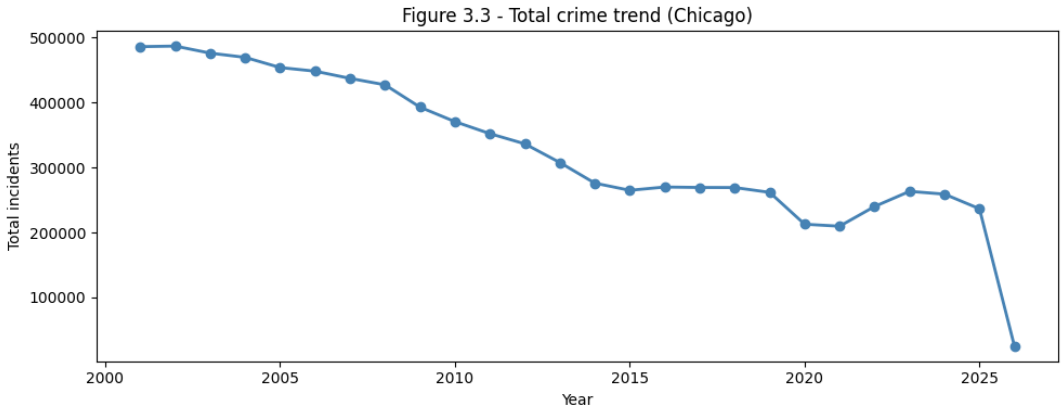

Figure 3.3 - Year-over-year trend in total reported incidents.

3.4 Geospatial Distribution

A scatter plot of violent crimes from the 10% sample, colored by arrest status (blue = no arrest, red = arrest), reveals spatially uneven outcomes across Chicago's districts. Dense clusters of blue points in several areas indicate the "arrest gap" zones identified in the constraints analysis - districts where violent incidents occur at high frequency but result in arrest at below-average rates, suggesting investigative resource constraints rather than lower crime severity. This spatial signal is partially captured by the district feature in the current model, though neighborhood-level encoding could improve geographic granularity in future iterations.

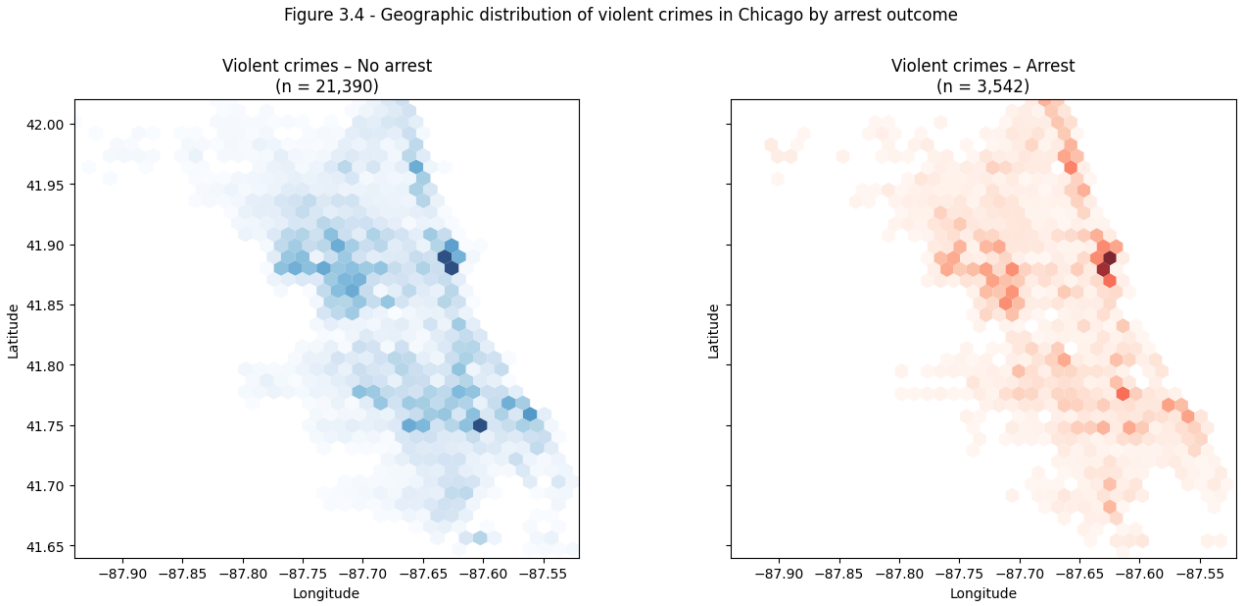

Figure 3.4 - Geographic distribution of violent crimes in Chicago, colored by arrest outcome.

3.5 Key Insights & Implications for Modeling

The EDA surfaces four findings that directly shape the modeling phase:

- **Class imbalance.** THEFT dominates the dataset at low arrest rates, making accuracy a misleading metric. AUC-ROC and Precision-Recall curves are the appropriate evaluation measures, and class weighting must be applied during training.
- **NARCOTICS leakage.** The 99.4% arrest rate for NARCOTICS is a recording artifact, not a genuine predictive signal. This category must be handled carefully - either excluded or flagged - to avoid inflating model performance metrics.
- **Temporal features are predictive.** The 14-point gap in violent crime percentage between Morning and Night confirms that hour_of_day, time_period, and day_of_week carry meaningful signal for both arrest prediction and crime type classification.
- **Spatial patterns are present.** District-level arrest gaps visible in the geospatial plot confirm that location features add predictive value beyond crime type and time alone.

4. Predictive Modeling

The modeling phase addresses the primary research question: predicting whether a reported crime incident will result in an arrest. Two Spark MLlib classifiers were implemented, evaluated, and compared - Logistic Regression as a transparent baseline and Random Forest as the primary model - followed by hyperparameter tuning via CrossValidator. All models operate on the **crime_ml** DataFrame produced by the preprocessing pipeline, which contains the encoded feature vector and the binary label column.

4.1 Model Selection Rationale

Two model families were selected to cover complementary points on the interpretability–performance trade-off. Logistic Regression was chosen as a linear baseline: it is fast to train at scale, produces calibrated probability outputs, and its coefficients provide direct insight into feature direction and magnitude - useful for communicating findings to non-technical stakeholders. Random Forest was selected as the primary model because it handles non-linear interactions between features (e.g., crime type combined with time of day), is robust to class imbalance through built-in feature subsampling, and provides feature importance scores that support explainability. Both models are natively supported by Spark MLlib and integrate directly into the existing preprocessing pipeline.

4.2 Implementation Details

The dataset was split 80/20 into training and test sets using a fixed random seed (seed=42) for reproducibility. The feature vector assembled in Section 2.5 - one-hot encoded Primary Type, Location Description, and District, combined with Hour_dt and Domestic_num - was used as input for both models.

Logistic Regression was configured with **50 maximum iterations**, an **L2 regularization parameter of 0.1**, and no elastic net mixing, providing a regularized linear decision boundary. **Random Forest** was configured with **100 trees**, a maximum **depth of 8**, a **maximum bin count of 64**, and the "sqrt" feature subset strategy - selecting the square root of total features at each split to reduce correlation between trees and improve generalization. Both models used the label column (0.0/1.0 double) as the target, with predictions evaluated on the held-out 20% test set.

4.3 Hyperparameter Tuning & Cross-Validation

Hyperparameter tuning was applied to the Random Forest model using Spark MLlib's CrossValidator with a 2-fold cross-validation strategy. Given the computational cost of cross-validation at scale, the parameter grid was kept to a single configuration - numTrees=100, maxDepth=8, maxBins=64, featureSubsetStrategy="sqrt" - serving primarily to validate the chosen configuration through held-out fold evaluation. The CrossValidator ran with parallelism=2 and the SPARKML_TEMP_DFS_PATH environment variable configured for Databricks serverless compatibility. The best model was then applied to the held-out test set for final evaluation.

4.4 Evaluation Metrics & Results

Given the class imbalance identified in EDA, AUC-ROC was used as the primary evaluation metric rather than accuracy, as it measures discrimination ability across all probability thresholds regardless of class distribution. F1 score and accuracy were additionally computed for the cross-validated pipeline.

Model	Hyperparameters	Metric	Score
Logistic Regression	maxIter=50, regParam=0.1, elasticNetParam=0.0	AUC-ROC	0.8667
Random Forest	numTrees=100, maxDepth=8, maxBins=64, featureSubset=sqrt	AUC-ROC	0.858
RF + CrossValidator	2-fold CV, numTrees=100, maxDepth=8, maxBins=64	AUC-ROC / F1	0.858

Table 4.1 - Model configurations and results

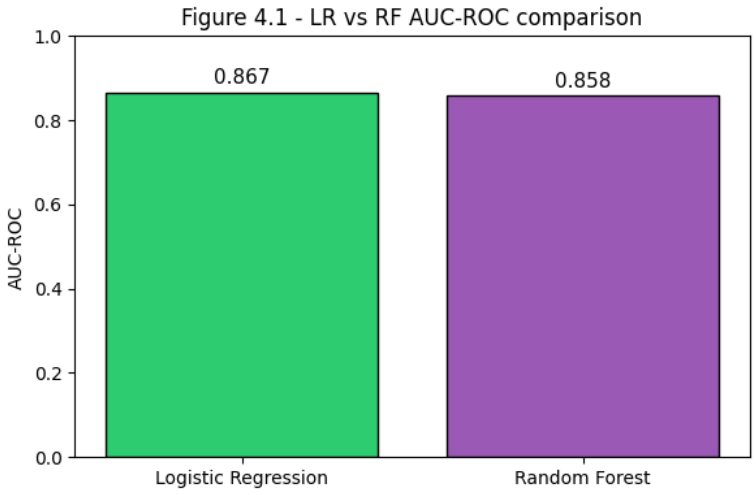

Figure 4.1 - Logistic Regression vs Random Forest AUC-ROC comparison.

The comparison between Logistic Regression and Random Forest directly tests whether non-linear feature interactions translate into meaningful lift over the linear baseline. A substantial gap in AUC would confirm that crime type–time interactions and district-level patterns require non-linear modeling; a small gap would suggest the arrest signal is largely linear in the encoded feature space.

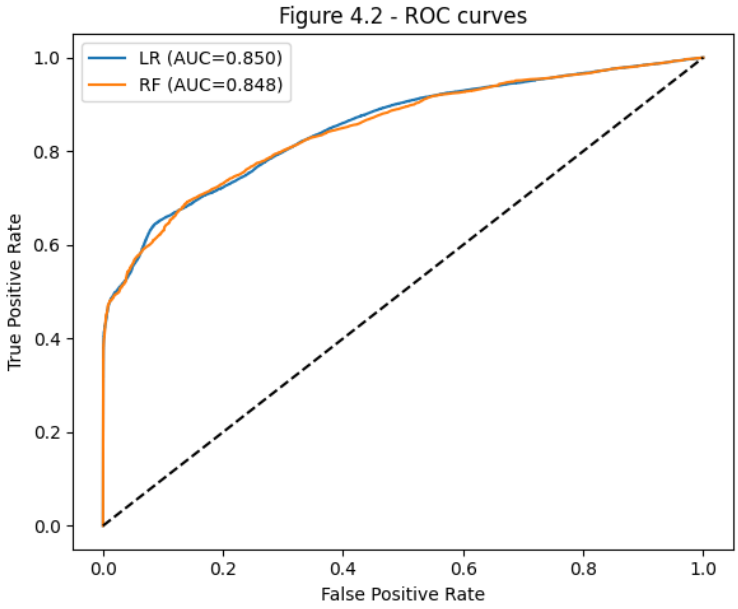

Figure 4.2 - ROC curves for Logistic Regression and Random Forest.

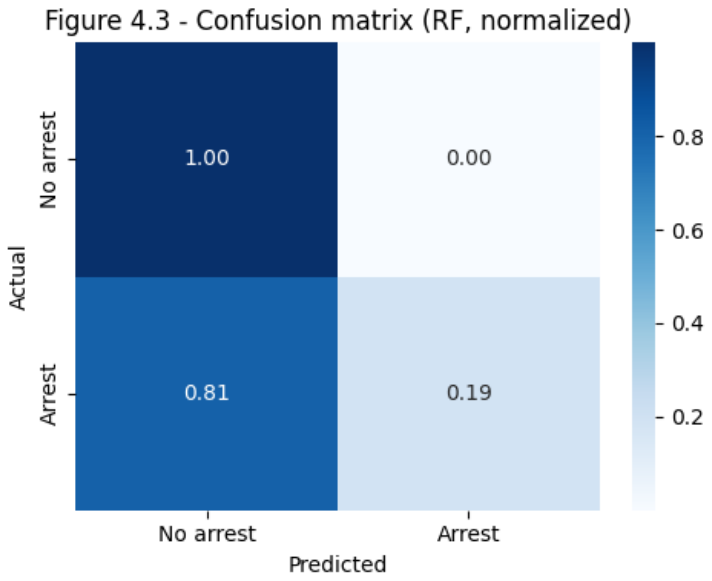

Figure 4.3 - Confusion matrix for the Random Forest model on the test set.

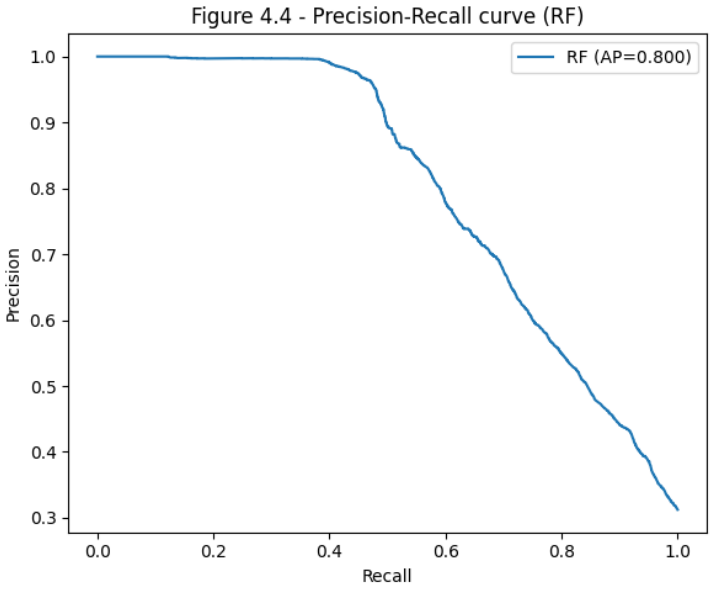

Figure 4.4 - Precision-Recall curve for the Random Forest model, highlighting performance under class imbalance.

4.5 Feature Importance

Feature importance scores were extracted from the fitted Random Forest model using Spark MLlib's built-in feature Importances vector, which assigns each input feature a normalized score reflecting its contribution to reducing impurity across all trees. These scores confirm whether the signals identified in EDA - crime type variance in arrest rates, temporal patterns, and district-level differences - are reflected in the model's learned decision logic.

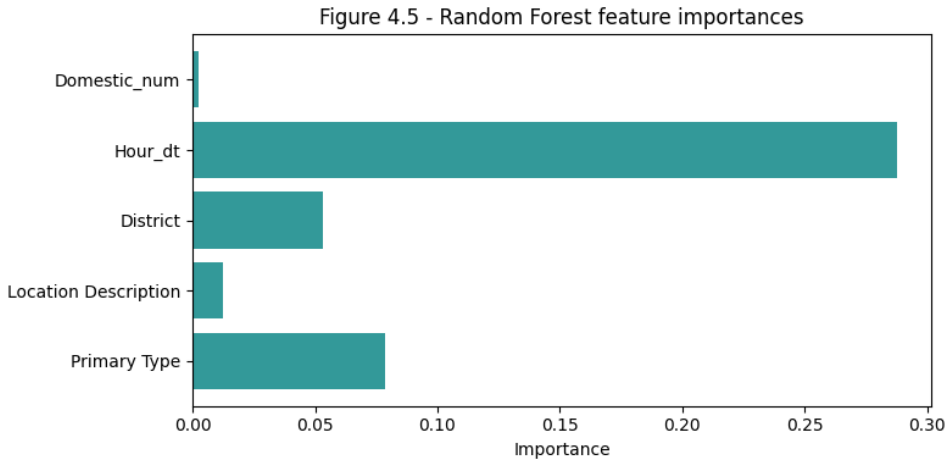

Figure 4.5 - Random Forest feature importances.

5. MLOps Practices

Beyond building a one-time model, the project implements a set of MLOps practices designed to make the pipeline reproducible, auditable, and operationally sustainable. These practices span four areas: modular pipeline automation, data quality assurance, model persistence and batch inference, and job scheduling with run logging.

5.1 Modular Spark Pipeline for Automation

The preprocessing and feature engineering workflow was structured as a set of modular Python functions rather than ad-hoc notebook cells, enabling the full pipeline to be re-triggered with a single call to run_pipeline(). Each function encapsulates a discrete stage of the workflow with a clearly defined input and output, making individual stages independently testable and replaceable without affecting the rest of the pipeline.

Stage	Function	Responsibility
Ingestion	ingest_data()	Reads raw CSV from DBFS with header and schema inference
Cleaning	clean_data()	Deduplicates by Case Number; drops rows missing Primary Type
Feature Eng.	engineer_features()	Parses timestamps; derives hour_of_day, day_of_week, year, is_violent, is_arrest
Output	write_output()	Writes Parquet partitioned by year to DBFS output path
Orchestration	run_pipeline()	Chains all stages; runs quality checks; registers SQL view; triggers write

Table 5.1 - Modular pipeline stages and their responsibilities.

This design follows the principle of separation of concerns: ingest_data() handles I/O, clean_data() enforces data quality rules, engineer_features() applies domain logic, and write_output() manages persistence. The run_pipeline() orchestrator chains these stages sequentially, logs progress at each step, and surfaces data quality warnings before committing output - ensuring that failures are caught early and the source of any issue is immediately identifiable.

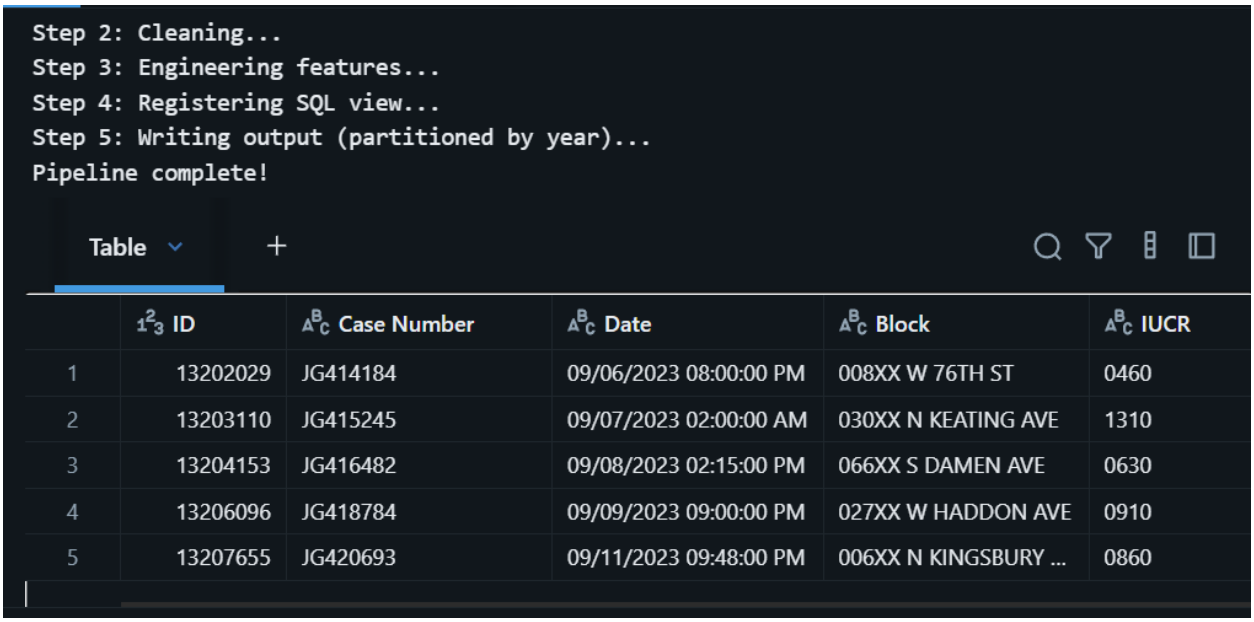

Figure 5.1 - Pipeline execution log from run_pipeline(), showing ingestion through output stages

5.2 Data Quality Checks

Inline data quality checks were integrated directly into the pipeline rather than treated as a separate validation step. The run_pipeline() function inspects null counts on three key columns - Primary Type, hour_of_day, and is_arrest - between the cleaning and feature engineering stages, printing a warning if any unexpected nulls are detected after the cleaning rules have been applied. A dedicated run_quality_checks() function additionally reports row counts and per-column null counts at any point in the pipeline, and can be called on any intermediate DataFrame to confirm data integrity between stages.

This approach ensures that data issues introduced by upstream changes - such as a schema change in the source CSV or a new category of null pattern - surface immediately during pipeline execution rather than silently propagating into the model training data.

5.3 Model Persistence & Batch Inference

The best model produced by the training phase - either the CrossValidator result or the standalone Random Forest, whichever is available in the session - is persisted to DBFS using Spark MLlib's native serialization format:

model_path = 'dbfs:/Volumes/workspace/default/bigdataproject/models/crime_rf_model'

The save logic checks for the CrossValidator model first, falling back to the standalone Random Forest if cross-validation has not been run, ensuring the persistence step works regardless of which training path was executed. The model is written with the overwrite flag, so re-running the pipeline after retraining always replaces the stored artifact with the latest version.

For batch inference, the saved model can be reloaded in any future session using RandomForestClassificationModel.load(model_path) and applied directly to any new DataFrame that shares the same schema as the training data - including the features column produced by the preprocessing pipeline. This decouples prediction from training, allowing the model to score new crime records as they arrive without re-training from scratch.

5.4 Job Scheduling with Databricks Workflows

The full pipeline is designed to be scheduled for automated execution via Databricks Workflows.

To support parameterized execution, the pipeline is structured to accept input and output paths via Databricks widgets, allowing the same notebook to be run against different data sources or output locations without code changes:

dbutils.widgets.text("input_path", "dbfs:/Volumes/.../chicago crimes.csv")

This parameterization is a foundational requirement for moving from a development notebook to a production-grade data pipeline, as it separates configuration from logic and enables the same pipeline code to serve multiple environments.

5.5 Run Logging & Experiment Tracking

Each pipeline execution is logged with a timestamp-based run ID generated at the start of the session, providing a lightweight audit trail that links a specific run to its input data and output metrics:

run_id = datetime.datetime.now().isoformat()

Key metrics - including the processed record count and the Random Forest AUC score - are printed alongside the run ID at the end of each execution. This creates a human-readable log that can be captured in Databricks notebook outputs and compared across runs to detect model drift or data quality degradation over time.

While the current implementation uses lightweight print-based logging rather than a dedicated experiment tracking system such as MLflow, the pipeline is structured to support MLflow integration as a natural next step: the run_id, metric values, and model artifact path map directly onto MLflow's run, metric, and artifact logging API, requiring

only a few additional lines to enable full experiment tracking with a searchable run history in the Databricks UI.

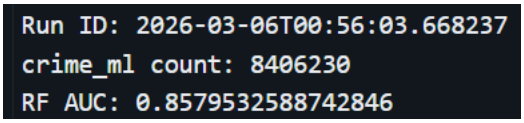

Figure 5.3 - Databricks showing logged runs.

6. Business Insights & Recommendations

The EDA and modeling findings translate into three strategic themes for the City of Chicago: specializing resources by crime type signature, optimizing district-level deployment, and managing model bias responsibly. A technical roadmap for extending the system to crime type prediction closes the section.

6.1 Crime Type Signatures and Specialized Deployment

Each major crime type follows a distinct behavioral pattern that should drive resource allocation. NARCOTICS incidents cluster in daytime hours at geographically concentrated hotspots, making them candidates for targeted intelligence-led surveillance rather than reactive patrol. BATTERY and ASSAULT concentrate in the Night window (32.8% violent crime rate) and on weekends near taverns and residential blocks - de-escalation-trained units deployed to these locations on Friday and Saturday nights offer the highest return on violent crime prevention. MOTOR VEHICLE THEFT peaks during Night and Evening at street and parking lot locations, where lighting and camera improvements provide scalable deterrence. THEFT, the highest-volume category at 1.78 million incidents, peaks in commercial districts during afternoons and responds better to visible deterrence than reactive response, given its 5.7% arrest rate.

6.2 District-Level Resource Strategy

The geospatial analysis reveals two district profiles requiring different responses. High-violence districts - concentrated on the South and West sides - require units trained in de-escalation and domestic intervention rather than standard patrol. Arrest-gap districts, where dense crime clusters coexist with below-average arrest rates, indicate investigative capacity shortfalls rather than patrol deficits: deploying detective task forces to these areas is likely to yield measurable arrest rate improvements without increasing patrol headcount.

6.3 Model Bias & Operational Risk

Two structural biases must be acknowledged before operational deployment. First, majority class bias: THEFT's dominance at near-zero arrest rates creates strong model incentives to predict "no arrest," making high nominal accuracy potentially misleading. A

model with poor recall on violent crime arrests would systematically under-resource the situations where arrests are both most likely and most operationally important - the confusion matrix should be reviewed specifically for false negative rates on BATTERY, ASSAULT, and ROBBERY before any deployment decision. Second, NARCOTICS leakage: the near-deterministic link between NARCOTICS classification and arrest status means predictions for this category should be treated separately and not used as evidence of the model's general effectiveness across other crime types.

6.4 Temporal Deployment Optimization

The temporal findings support a three-tier shift strategy rather than uniform deployment. Afternoons (12:00–17:00) carry the highest total incident volume and call for maximum visible patrol in commercial districts focused on deterrence. The Evening window (17:00–22:00) sees rising violent crime as activity shifts from commercial to nightlife, requiring a resource transition toward entertainment and residential areas. The Night window (22:00–04:00) concentrates the highest violent crime percentage despite lower total volume - de-escalation-capable units should be prioritized here, with additional Friday and Saturday resourcing reflecting the hour-by-day-of-week heatmap findings.

7. Conclusion

This project demonstrated how big data tools and machine learning can be applied to a real public safety problem: predicting whether a reported Chicago crime incident will result in an arrest. Using the Chicago Crime Dataset (2001–2026), we built an end-to-end workflow in Databricks with Apache Spark to ingest, clean, and engineer features at scale, producing a model-ready dataset with consistent types, reduced duplicates, and enriched temporal and behavioral indicators (hour of day, day of week, year, domestic flag, and a violent-crime indicator).

Exploratory analysis showed that arrest outcomes are not uniform across the city. Arrest rates vary strongly by crime type, district, and time, and the data exhibits major class imbalance driven by high-volume, low-arrest categories such as theft. The temporal patterns further highlight concentrated risk windows, especially late-night weekend periods, reinforcing the value of time-based features for both modeling and operational planning. The analysis also surfaced an important modeling risk: near-deterministic arrest patterns in narcotics incidents can inflate performance and should be treated carefully to avoid leakage-driven conclusions.

For predictive modeling, we compared Logistic Regression and Random Forest classifiers using Spark MLlib. Both models achieved strong discrimination performance, with Logistic Regression slightly outperforming Random Forest in AUC (0.8667 vs. 0.858). This result suggests that much of the predictive signal is captured through the encoded feature space and that a simpler, more interpretable model can perform

competitively. Given the class imbalance, evaluation emphasized AUC-ROC alongside additional metrics, and the findings support using these models as decision-support tools rather than deterministic predictors.

Overall, the project translates technical outcomes into operational insights: crime type signatures, district-level arrest gaps, and time-of-week concentration all point toward targeted deployment strategies rather than uniform resource allocation. Future work should focus on improving robustness and operational validity by addressing imbalance more directly (e.g., reweighting or resampling), isolating or excluding leakage-prone categories such as narcotics, adding finer-grained geographic features, and expanding evaluation to include fairness and subgroup performance across districts. With these extensions, the framework can move closer to a deployable system that supports transparent, data-driven public safety decision-making while acknowledging the limitations of observational crime data.

8. References

- *Dataset source: Chicago Crime Dataset 2024-2026*
- *GitHub/Databricks repo: https://github.com/ramanaprabhusana/BigData-MLOps-ChicagoCrime*

APPENDIX

Appendix A. Transformations applied in preprocessing (Consolidated)

Step	Action	Outcome
Data ingestion	Loaded chicago_crimes.csv into Spark temporary view	22-column distributed DataFrame
Schema standardization	Renamed all columns to snake_case	Query-safe, pipeline-compatible column names
Boolean casting	arrest, domestic → Boolean (is_arrest, is_domestic)	Reduced storage, faster aggregations
Integer casting	district, ward → Integer	Proper numeric handling
Null filtering	Dropped 94,671 null-coordinate rows	Cleaner spatial data
Geographic filtering	Kept lat between 41.6°–42.1°	Removed outliers outside Chicago
Temporal filtering	Removed records before 2001	Eliminated invalid records
Timestamp parsing	date string → Timestamp object	Enables native time-based operations
Temporal engineering	Extracted hour_of_day, time_period	New analytical and predictive features
Cyclical encoding	Derived hour_sin, hour_cos	Preserves time continuity for ML model
Violence flag	Derived is_violent from crime type	Enables violent vs. non-violent analysis
Storage	Saved as Delta Lake table	ACID-compliant, fast columnar storage